

MySQL Assignment

Question 1: Installing MySQL Workbench & Viewing Default Schemas

Install MySQL Workbench on your computer, connect to the local MySQL server, and take a screenshot of the default databases shown in the Schemas panel.

Note: This step requires installing software on your own computer and capturing a screenshot from your own screen, which isn't something that can be done from here. Below is a written walkthrough of exactly what to do and what you should expect to see, so you can complete this step yourself and take the screenshot.

1. **Download MySQL Workbench** from the official site: dev.mysql.com/downloads/workbench/, choosing the installer for your operating system (Windows/macOS/Linux).
2. **Install it**, along with MySQL Server if it isn't already installed (MySQL Workbench is just the GUI client; MySQL Server is the actual database engine it connects to).
3. **Open MySQL Workbench** and click on the local connection tile (usually labeled something like `Local instance MySQL80`) under "MySQL Connections" on the home screen.
4. **Enter your root password** when prompted, to connect to the local server.
5. **Look at the "Schemas" panel** on the left-hand sidebar (below the Navigator). By default, MySQL ships with a few built-in schemas such as: `information_schema`, `mysql`, `performance_schema`, and `sys`. These are system databases used internally by MySQL and are present on every fresh installation.
6. **Take the screenshot** of this Schemas panel showing these default databases listed (use your OS's screenshot tool — Snipping Tool/Win+Shift+S on Windows, Cmd+Shift+4 on macOS).

What you should see: The Schemas panel, once expanded, will display the four default system schemas mentioned above in a tree list, each with a small database icon next to it. No user-created databases will appear yet since none have been created — that happens in Question 2.

Question 2: Creating the InstaClone Database and Users Table

Create a new database called InstaClone in MySQL Workbench, then create a table named Users with columns: `user_id` (INT, primary key), `username` (VARCHAR), `email` (VARCHAR), and `followers_count` (INT).

create_users_table.sql

```
-- Step 1: Create the database
CREATE DATABASE InstaClone;

-- Step 2: Select it so subsequent statements run inside it
USE InstaClone;

-- Step 3: Create the Users table
CREATE TABLE Users (
  user_id      INT PRIMARY KEY,
  username     VARCHAR(50) NOT NULL,
  email        VARCHAR(100) NOT NULL,
```

```
followers_count INT DEFAULT 0
);
```

Explanation: `CREATE DATABASE InstaClone;` creates a new empty schema, and `USE InstaClone;` switches the active context to it so the following `CREATE TABLE` statement runs inside that database. The `Users` table defines `user_id` as an `INT` and marks it `PRIMARY KEY`, meaning each value must be unique and non-null, uniquely identifying every row. `username` and `email` use `VARCHAR` for variable-length text, and `followers_count` is an `INT` with a `DEFAULT 0` so new users start with zero followers unless specified otherwise.

Question 3: Inserting Sample Users

Insert 3 sample users into the `Users` table you created, using realistic Instagram-style usernames and follower counts.

`insert_users.sql`

```
INSERT INTO Users (user_id, username, email, followers_count)
VALUES
  (1, 'travel.with.riya', 'riya.sharma@example.com', 24500),
  (2, 'foodie_karan', 'karan.mehta@example.com', 8900),
  (3, 'fit_with_aisha', 'aisha.khan@example.com', 152300);
```

Explanation: A single `INSERT INTO ... VALUES` statement is used to add three rows at once, each wrapped in its own parentheses and separated by commas. The usernames follow common Instagram naming patterns (lowercase, dot/underscore-separated, niche-themed like travel, food, and fitness), and the `followers_count` values vary realistically to represent different levels of account popularity.

Question 4: Creating the Posts Table with a Foreign Key

Create another table in the `InstaClone` database called `Posts` with columns: `post_id` (`INT`, primary key), `user_id` (`INT`), `caption` (`VARCHAR`), and `post_date` (`DATE`). Add a foreign key from `Posts.user_id` to `Users.user_id` to establish a relationship.

`create_posts_table.sql`

```
CREATE TABLE Posts (
  post_id    INT PRIMARY KEY,
  user_id    INT,
  caption    VARCHAR(255),
  post_date  DATE,
  FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- Example inserts to demonstrate the relationship
INSERT INTO Posts (post_id, user_id, caption, post_date)
VALUES
  (101, 1, 'Sunset views from the mountains 🌄', '2026-06-12'),
  (102, 2, 'Best butter chicken in town 🍛', '2026-07-03'),
  (103, 3, 'Leg day done 💪', '2026-08-19');
```

Explanation: The `Posts` table stores each post's own `post_id` as its primary key, along with a `user_id` column that identifies which user made the post. The line `FOREIGN KEY (user_id)`

REFERENCES `Users(user_id)` establishes a relationship between the two tables: it tells MySQL that every value in `Posts.user_id` must already exist as a `user_id` in the `Users` table. This enforces referential integrity — for example, you cannot insert a post for a `user_id` that doesn't exist in `Users`, and by default MySQL will also prevent deleting a user who still has posts linked to them (unless cascading rules are added).

Question 5: Comparison of MySQL, PostgreSQL, Oracle, and SQLite

Write a short comparison (3-4 lines each) of MySQL, PostgreSQL, Oracle, and SQLite, focusing on where each one is commonly used (for example: web apps, mobile apps, enterprise systems, etc.).

Database	Common Use Cases
MySQL	MySQL is one of the most widely used open-source relational databases, especially popular for web applications. It powers a huge share of content management systems (like WordPress), e-commerce platforms, and general-purpose web backends because it is free, fast for read-heavy workloads, and has broad hosting support. It's a common default choice for startups and small-to-medium web projects due to its simplicity and large community.
PostgreSQL	PostgreSQL is an open-source relational database known for strict standards compliance, advanced data types (JSON, arrays, geospatial data via PostGIS), and strong support for complex queries and transactions. It's commonly used in applications requiring high data integrity, analytics, and complex business logic — such as fintech, SaaS platforms, and data-heavy backend systems. Many modern startups prefer it over MySQL for its extensibility and correctness.
Oracle Database	Oracle Database is a commercial, enterprise-grade relational database known for its robustness, scalability, and advanced features like partitioning, high availability, and enterprise security. It's the backbone of many large corporations, banks, telecom companies, and government systems that need to handle massive transaction volumes with strict uptime and compliance requirements. It's less common in small projects due to its licensing cost and complexity.
SQLite	SQLite is a lightweight, serverless, file-based database engine that requires no separate database server process. It's the default choice for mobile apps (both Android and iOS use it internally), embedded systems, browser storage, and small desktop applications where a full client-server database would be overkill. It's not designed for high-concurrency, multi-user server environments, but excels at local, single-user or low-traffic data storage.